COMP 138 RL: Homework 1

Longji Li

February 15, 2025

1 Introduction

The k-armed bandit problem is a classic reinforcement learning problem that an agent chooses from k different actions or "arms", each with an unknown reward distribution. The objective of the problem is to maximize cumulative reward over time by balancing exploration and exploitation. In this work I designed a 10-armed bandit problem, an agent will repeatedly select one of 10 arms and receive a stochastic reward based on normal distribution to investigate the performance of different balancing methods in nonstationary problems.

2 Background

2.1 Nonstationary Problem

The reward distributions of actions will remain fixed in stationary problem. However, in nonstationary problem, the reward distributions of actions change over time. For example, in a casion, the reward of a bandit might be adjusted periodically.

2.2 Sample-average Method

To estimate the value of Q_{n+1}, the sample-average method computes the average of all observed rewards for that action:

$$\begin{aligned} Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\ &= Q_n + \frac{1}{n}[R_n - Q_n] \end{aligned} \tag{1}$$

where R_i is the reward after ith selection of the arm and Q_n is the estimated value of the action after $n-1$ selections.

2.3 Constant Step Size Method

The constant step size method updates the average Q_n using constant a, which gives more weight to recent rewards

$$Q_{n+1} \doteq Q_n + \alpha[R_n - Q_n]$$

where $a \in (0, 1]$.

2.4 $\epsilon-$greedy action-value methods

This method uses greedy algorithm that selects the arm with the highest estimate value $Q(a)$ most of time. However, with ϵ probability, it will select a random arm instead.

2.5 Softmax Action Selection

The Softmax method is another action selection strategy that assigns a probability to each action based on its estimated value. This allows actions with higher expected rewards to be chosen frequently and it also allows to explore of lower-valued actions.

2.6 Upper-Confidence-Bound Action Selection

The Upper-Confidence-Bound (UCB) algorithm is an action selection method in reinforcement learning that balances exploration and exploitation in an efficient way. Unlike $\epsilon-$greedy method, it also takes into account the uncertainties in actions.

3 Experiment

To evaluate the performance of the sample-average and constant step-size methods in nonstationary environments, we conduct an experiment using a simulated 10-armed bandit problem. I believe, in this 10-armed bandit problem, the constant step-size method ($\alpha = 0.1$) will outperform the sample-average method in terms of tracking the optimal action and maximizing average reward over time, as the sample-average methods update the action-value estimate using the mean of all past rewards, which causes slow adaptation to nonstationary reward distributions.

3.1 Experiment Setup

As we need to design a 10-armed bandit, we set the number of actions equals to 10. And each action's reward follows a Gaussian distribution, and the true action values evolve over time following a random walk

$$q_*(a) \leftarrow q_*(a) + \mathcal{N}(0, 0.01)$$

which simulates a nonstationary reward distribution. We run the individual experiments 2000 times and 10000 time steps in each run. The learning strategies we use in this experiment are sample-average method and constant step-size method (where $\alpha = 0.1$) as we mentioned in the introduction. For the action selection strategy, we implement ϵ-greedy policy with $\epsilon = 0.1$. In doing this, the agent use greedy selection in 90% of time and randomly select arms in 10% of time.
In order to evaluate the performance of the two strategies, we use the average reward and percentage of optimal action selections to measure learning performance.

4 Result

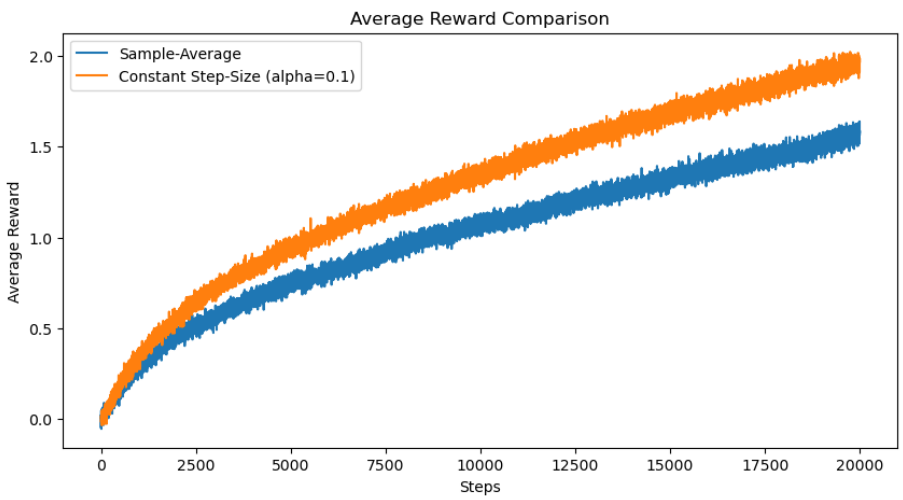

Figure 1: The agent's performances under different strategies.

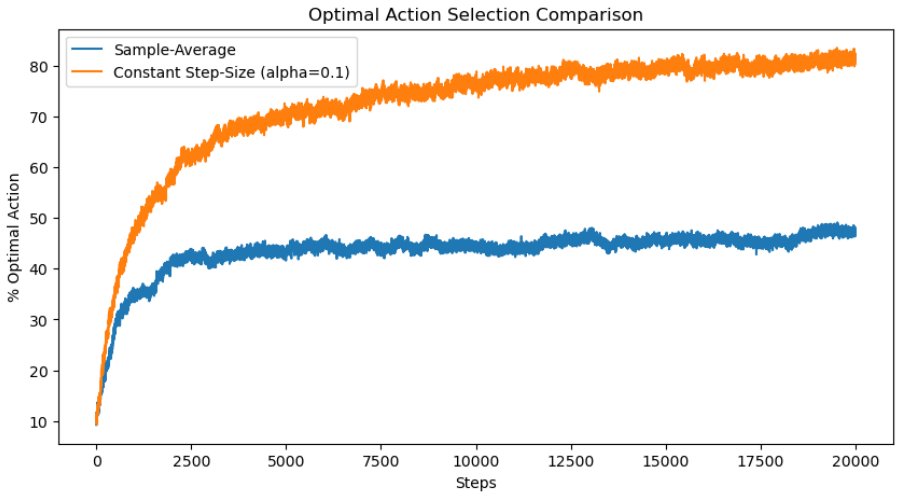

Figure 2: he agent's optimal action selections under in different strategies. In the first 2500 time steps, are learning the choose the optimal actions. The rest of time steps, the agent are stabilizing at a percentage of optimal selections

4.1 Discussion

Based on the result, we find that the agent using constant step-size methods ($\alpha = 0.1$) performs better than it using sample-average methods. At the end of the time step, the average reward of sample-average methods achieves 2.0, and, in contrast, the average reward of sample-average is around 1.5. This means that the agent learns quickly when we use constant α, which has a higher average reward in the period of the first 20000 time steps. In addition, the result of optimal action selection comparison (fig.2) shows that the sample-average method in nonstationary situation has a lower percentage of optimal action selection around 43%. However, the constant step-size ($\alpha = 0.1$) has a better performance better in selecting the optimal action. It tracks the optimal action more effectively, maintaining a higher selection percentage over 75% after 2500 time steps. Therefore, the constant step-size methods seem to have a better performance in nonstationary problem.

5 Additional Question

In Part 1, we find that the constant step-size method ($\alpha = 0.1$) performed better than the sample-average method in a nonstationary 10-armed bandit problem, as it adapted more quickly to changing reward distributions. However, different action selection methods may also be able to affect agent's performance. How does different action selection strategies affect the learning performance?

5.1 Experiment

In this part, we applied three different action selection methods, including $\epsilon-$greedy ($\epsilon = 0.1$), upper-confidence-bound action selection (UCB), and Softmax. We investigate how different action selection strategies influence the performance of the constant step-size method in a nonstationary 10-armed bandit setting. By comparing these approaches, we aim to explore which strategy leads to higher average rewards and better tracking of the optimal action over time.

5.2 α Selection?

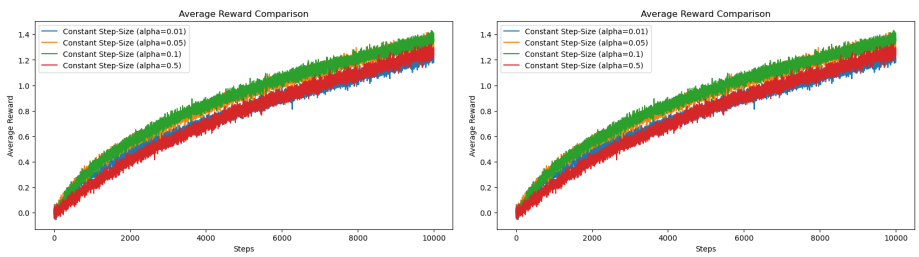

Figure 4: The agent's performance in different α.

Before the experiment, we test different parameter α, and we find that $\alpha = 0.1$ has the best performance on both average reward and optimal action exploration. Therefore, we use $\alpha = 0.1$ in the following experiment.

5.3 Experiment Setup

In this problem, I believe that UCB or Softmax will perform better than $\epsilon-$greedy. UCB selects actions based on both the estimated value and the uncertainty (variance) of the arms, which allows it to explore actions that have been selected less frequently. Similarly, Softmax method assigns probabilities to actions based on their estimated values, which ensures a more balanced trade-off between exploration and exploitation.

To conduct this addition experiment, we modified the 10-armed bandit problem by removing the sample-average method and adding the three a action selection strategies in constant step-size method ($\alpha = 0.1$). In addition, the Temperature in Softmax we use $\tau = 1$ and the confidence parameter in UCB we use c = 2. We run the individual experiments 2000 times and 10000 time steps in each run.

5.4 Result

Figure 4: The average rewards in different action selection strategies.

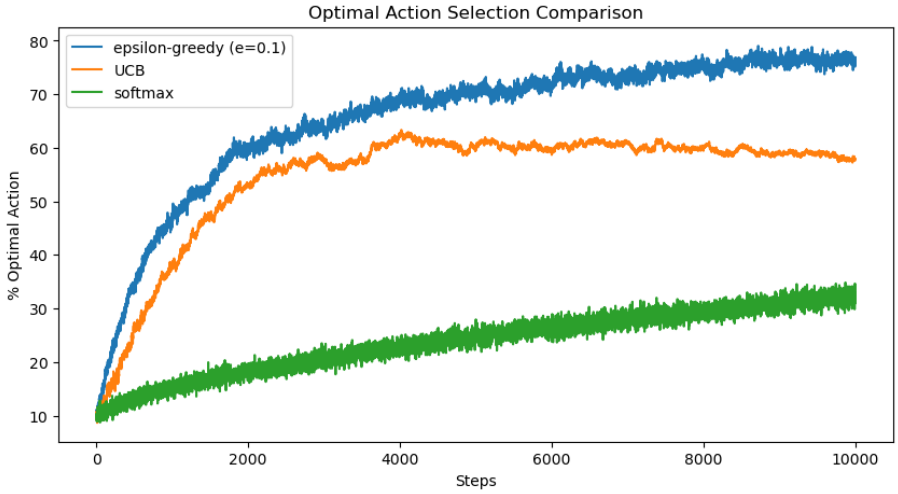

Figure 5: The percentage of optimal action in different action selection strategies.

5.5 Discussion

In this experiment, we only apply the default parameters, including $\epsilon = 0.1$,$\tau = 1$,and $c = 2$. The results were analyzed based on average reward accumulation and optimal action selection percentage over 10,000 time steps. In figure 4, it illustrates that $\epsilon-$greedy and UCB achieve similar average rewards, particularly between steps 2,000 and 7,000. However, $\epsilon-$greedy ultimately shows slightly

higher rewards after 8000 steps. In figure 5, it shows that $\epsilon-$greedy selects the optimal action at a higher rate around 70–80% over time, which demonstrates that it balance exploration and exploitation effectively. However, UCB seems has trend of decrement after 4000 step. Softmax method in this experiment shows a lowest average reward and percentage of optimal actions, which indicates that a temperature parameter of $\tau = 1$ results in excessive exploration, which means it prevents effective convergence to the optimal action.
In conclusion, $\epsilon-$greedy is the most effective method in this experiment setting, as it efficiently identifies and exploits the optimal action while maintaining an appropriate level of exploration. The UCB method performs well in average reward but shows a lower optimal action selection percentage. The Softmax method shows the lowest result in both average reward and optimal action selection, meaning that we should adjust the temperature to improve the performance.

6 Limiation

The experiments conducted in Part 1 and Part 2 are performed under a limited setting. For instance, in Part 2, we evaluate each method using only a single hyperparameter configuration. This constraint may lead to results that are representative of only a specific scenario, rather than providing a comprehensive evaluation across different parameter choices. As a result, the conclusions drawn may be influenced by the chosen hyperparameters, and further experimentation with a larger range of values could provide more generalizable conclusion.

7 Conclusion

In part1, by comparing the sample-average method with constant step-size method using 10-armed bandit, we find that constant step-size methods have better performance in balancing balancing exploration and exploitation in non-stationary problem. Moreover, in the part2, we conduct an additional experiment which aims to find the best action selection methods. It shows that $\epsilon-$greedy method are better than the other methods. To explore this with more accuracy, we need to test more different values of parameters in the future.

8 Extra Credits

8.1 Exercise 3.6

Consider the Pole-Balancing problem and treat the it as an episodic task using discounting, the return will be

$$G_t = 0 + 0 + ... + 0 + (-\gamma^{K-1}) = -\gamma^{K-1}$$

as the reward would be -1 on failure and zero at all other time steps. However, if we treat it as a continuing task, the return will be

$$G_t = -1 - \gamma - \gamma^2 - \gamma^3 - ... = -\frac{1}{1-\gamma}$$

.

8.2 Exercise 3.7

The agent does not improve because it probably get the +1 reward in extremely rare situation, which means the maze might be complex and the robot only escape few times during the learning. They don't not effectively communicated the goal because most time the robot just receive 0 reward rather than a positive reward, which cause the gradient calculated from the reward is extremely small so that the robot will learn slowly rather than quickly escape from maze.

References

[1]R. S. Sutton and A. Barto, Reinforcement learning : an introduction. Cambridge, Ma ; London: The Mit Press, 2018.